

数据可视化

Data Visualization

清华大学统计与数据科学系 · 俞声 · 2026.5.13

1 数据可视化的两大目的

数据可视化并不只是“画图”，它在数据科学工作流中承担两种截然不同的角色：

1.1 探索性数据分析（EDA）

在建模与分析之前，我们需要先**理解数据本身的形状**：分布如何？有无异常值？类别间是否存在差异？变量之间有无关联？目视往往比统计摘要更直观。

- 目标受众**：是自己
- 风格要求**：快速、粗略，够用即可，不必追求美观
- 典型场景**：对每个特征画密度图，观察类别 0 和类别 1 的分布差异；画散点矩阵看多变量关系

EDA 实例：特征分布可视化

对同一特征（如 C0009274），分别绘制标准化前后的密度曲线，对比两个标签（0/1）的分布形状。

若两类分布重叠严重 → 该特征区分度低；若分离明显 → 该特征有较强预测力。

标准化后（Normal Mixture Normalization）分布形状改变，ROC 曲线 AUC 显著提升。

1.2 向受众传达分析结果

分析完成后，我们需要**把结论呈现给别人**：老板、客户、论文读者……此时图表必须清晰、自解释，且美观得体。

- 目标受众**：他人（无法实时提问）

- **风格要求：**标题、坐标轴标签、图例完整；颜色舒适；字体足够大
- **典型场景：**论文图、汇报 PPT 中的流程图、产品报告里的 KPI 仪表板

维度	探索性 EDA	传达成果
受众	自己	他人
速度	快速迭代	精心打磨
颜值	不重要	很重要
标注	可以省略	必须完整
工具	plt 快速接口	fig/ax 精确控制

2 Python 可视化工具全景

Python 生态中可视化工具众多，本讲义聚焦以下四种：

工具	定位	特点
matplotlib	基础绘图库	低层控制，灵活但繁琐；其他高层库的底层引擎
seaborn	统计绘图库	直接操作 DataFrame；内置分组（hue/style）；更好的默认样式
plotnine	ggplot2 移植版	基于图形语法，语法统一优雅；适合熟悉 R 的用户
Streamlit	交互式应用	快速将数据分析包装成 Web 应用，适合演示与共享

选择建议

- 快速探索 → matplotlib 的 plt 接口 或 seaborn
- 精确控制多子图 → matplotlib 的 fig/ax 接口
- 习惯 R 的 ggplot2 → plotnine（语法几乎一致）
- 做演示 Demo 或内部工具 → Streamlit

3 Matplotlib 基础

matplotlib 是 Python 中最基础的绘图库，几乎所有高层可视化库（seaborn、pandas 内置图等）都建立在其之上。理解 matplotlib 的机制，是掌握所有 Python 可视化的基石。

3.1 第一个图

导入惯例：`import matplotlib.pyplot as plt`，通常配合 NumPy 使用：

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 2*np.pi, 0.1) # 生成 0 到 2π 的等差数列
y = np.sin(x)

plt.plot(x, y) # 绘制折线图
plt.show()    # 渲染并显示
```

- `plt.plot(x, y)`：以 x 为横轴、y 为纵轴，绘制折线图
- `plt.show()`：将当前图形渲染出来并显示在屏幕上

Jupyter Notebook 注意事项

在 Jupyter 中，每个 cell 运行完毕后会自动调用 `plt.show()`，因此通常不需要手动调用。但有一个关键区别：`plt` 中的绘图状态会在 cell 结束时被重置，所以所有绘图命令必须写在同一个 cell 内。

3.2 show / close / figure 的区别

理解这三个函数的机制，是避免「为什么我的图多了一条线」或「内存占用过高」等常见困惑的关键。

`plt.show()`

将当前绘图中积累的数据和设置渲染成图像并显示。显示后，该图形从 pyplot 的「当前图形」中被冲刷（flush），但并不销毁内存中的对象。

为什么第二次 `savefig` 会出现两条线？

`plt` 始终在「当前图形」上叠加绘制。如果你调用 `plt.plot()` 后没有 `plt.show()` 或 `plt.close()`，

再次调用 `plt.plot()` 会将新曲线叠加到同一个图上，y 轴范围也会随之自动扩大。
这就是课件「Something Unexpected」例子的原因。

plt.close()

关闭当前图形并**释放其占用的内存**。下次调用 `plt.plot()` 时，会自动创建一个新的空白图形。

- **推荐场景：**在脚本中循环生成大量图形时，每次循环结束后调用 `plt.close()`，防止内存泄漏

plt.figure()

手动创建一个新的 Figure（画布）。此后所有绘图操作都落在这个新图形上，**但旧图形并不会被关闭**，只是不再是「当前图形」。

函数	作用	适用场景
<code>plt.show()</code>	渲染+显示，冲刷当前图	交互式环境（IPython/Jupyter）
<code>plt.close()</code>	关闭图形，释放内存	脚本批量生成图时，每张图结束后调用
<code>plt.figure()</code>	创建新画布（不关闭旧的）	需要同时维护多个独立图形时

```
# 脚本中的推荐写法
for i in range(100):
    plt.plot(x, np.sin((i+1)*x))
    plt.savefig(f'sin_{i+1}.png')
    plt.close()    # 每次循环必须关闭，否则内存不断累积
```

3.3 Figure 与 Axes 的解剖

matplotlib 的图形体系由两层组成：

- **Figure（画布）：**整张图的容器，包含背景、尺寸、保存格式等全局属性
- **Axes（绘图区）：**真正放数据的区域，含坐标轴、刻度、标签、标题、网格线等

一个 Figure 可以包含多个 Axes（即多子图）。每个 Axes 都有独立的坐标系。

```
# 方式1：先建 Figure，再添加 Axes
fig = plt.figure()
ax = fig.subplots()

# 方式2：一步到位（最常用）
fig, ax = plt.subplots()
```

```
# 绘图和标注
ax.plot(x, y)
ax.set_title('The Sine Function') # 注意: ax 用 set_title, plt 用 title
ax.set_xlabel('x')
ax.set_ylabel('y')
fig.savefig('my_plot.pdf')
plt.close(fig)
```

plt 接口与 ax 接口的对应关系

```
plt.title(...) ↔ ax.set_title(...)
plt.xlabel(...) ↔ ax.set_xlabel(...)
plt.ylabel(...) ↔ ax.set_ylabel(...)
plt.xlim(...) ↔ ax.set_xlim(...)
plt.ylim(...) ↔ ax.set_ylim(...)
plt.legend() ↔ ax.legend()
plt.grid() ↔ ax.grid()
```

3.4 快速接口 (plt) vs 面向对象接口 (fig/ax)

两种接口写出的图形完全相同，只是控制粒度不同：

快速接口 (plt)	面向对象接口 (fig/ax)
plt.plot(x, y)	ax.plot(x, y)
plt.title('The Sine Function')	ax.set_title('The Sine Function')
plt.xlabel('x')	ax.set_xlabel('x')
plt.ylabel('y')	ax.set_ylabel('y')
plt.savefig('my_plot.png')	fig.savefig('my_plot.png')
plt.close()	plt.close(fig)

背后机制：plt 每次调用时自动寻找「当前 Axes」并操作它。只有一个 Axes 时一切正常；一旦有多个子图，plt 就可能操作错误的子图，此时应使用 ax 精确指定。

使用 plt 接口的场景	使用 fig/ax 接口的场景
快速探索，交互环境	脚本中精确控制
每次只画一张图	多子图 (subplots)
不需要精细调整图形各部件	需要独立操作每个 Axes
代码简洁优先	可读性与可维护性优先

4 图形配置

4.1 图形尺寸

图形尺寸是 **Figure** 的属性，单位为英寸（inch），用 `figsize=(宽, 高)` 指定。

```
# — 三种设置方式 —————

# 方式1: 创建 Figure 时指定
plt.figure(figsize=(8, 6))    # 宽 8 英寸, 高 6 英寸
plt.plot(x, y)

# 方式2: 创建子图时指定（最常用）
fig, ax = plt.subplots(figsize=(6, 8))
ax.plot(x, y)

# 方式3: 事后修改
fig, ax = plt.subplots()
fig.set_size_inches(8, 6)
ax.plot(x, y)
```

默认尺寸是多少？

matplotlib 默认 `figsize = (6.4, 4.8)`，即宽 6.4 英寸、高 4.8 英寸（4:3 比例）。
论文通常要求单栏图宽度 ≈ 3.5 英寸，双栏 ≈ 7 英寸；汇报 PPT 可以用更大的尺寸。

4.2 全局配置 rcParams

matplotlib 的全局参数存储在 `plt.rcParams` 字典中，修改它会影响此后创建的**所有**图形。

```
# 修改默认图形尺寸
plt.rcParams['figure.figsize'] = (12, 6)

plt.plot(x, y)
plt.show()

# 恢复为 matplotlib 出厂默认值
plt.rcParams['figure.figsize'] = plt.rcParamsDefault['figure.figsize']
```

常用的 rcParams 键包括：

键	含义	示例值
figure.figsize	默认图形尺寸	(6.4, 4.8)
figure.dpi	默认分辨率	100
font.size	全局字体大小（点）	10
axes.titlesize	坐标轴标题字体大小	"large"
axes.labelsize	坐标轴标签字体大小	"medium"
lines.linewidth	默认线宽（点）	1.5
lines.markersize	默认标记大小（点）	6.0
axes.spines.top	是否显示顶部边框	True
axes.spines.right	是否显示右侧边框	True

4.3 保存图形

matplotlib 根据文件扩展名自动判断输出格式。格式分两大类：

- **栅格图（位图）**：PNG、JPG、BMP、TIFF。像素数 = 图形尺寸（英寸）× DPI，需要用 `dpi=` 控制清晰度
- **矢量图**：PDF、EPS、SVG。以数学曲线描述，无限缩放不失真，**推荐用于论文**（无需指定 DPI）

```
# — 保存栅格图（PNG，指定 DPI） —————
fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(x, y)
fig.savefig('my_plot.png', dpi=300)    # 300 DPI = 2400×1800 像素
plt.close(fig)

# — 保存矢量图（PDF，无需 DPI） —————
fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(x, y)
fig.savefig('my_plot.pdf')              # 无限放大不失真
plt.close(fig)

# — 其他常用参数 —————
fig.savefig('output.png',
            dpi=300,
            bbox_inches='tight',        # 裁掉多余空白（强烈推荐）
            transparent=True,          # 背景透明（适合 PPT 背景不为白色时）
)
```

5 plt.plot() 详解

plt.plot() 是 matplotlib 中使用频率最高的函数，可绘制折线图 and 散点图。它有极其丰富的参数，用于控制线型、标记、颜色、粗细等视觉属性。

5.1 线型 (linestyle / ls)

字符串	缩写	效果
'solid'	'-'	实线（默认）
'dashed'	'--'	虚线
'dotted'	'...'	点线
'dashdot'	'-.'	点划线
"	"	不画线（仅标记）

5.2 标记 (marker)

字符	形状	字符	形状
'.'	小圆点	'o'	圆形
's'	正方形	'P'	加号（粗）
'X'	叉号（粗）	'*'	星形
'D'	菱形	'd'	细菱形
'^'	上三角	'v'	下三角
'<'	左三角	'>'	右三角
'+'	加号（细）	'x'	叉号（细）
' '	竖线	'_'	横线

5.3 尺寸

线宽和标记大小的单位均为点（point），其中 1 point = 1/72 英寸。由于单位是相对于图形物理尺寸的，放大图形时线条/标记也会相应放大。


```
plt.plot(x, y, linewidth=3)          # lw= 是缩写, 默认 1.5
plt.plot(x, y, markersize=10)       # ms= 是缩写, 默认 6
plt.plot(x, y, marker='o', ms=10, lw=3)
```

5.4 颜色 (color)

matplotlib 支持多种颜色指定方式:

方式	示例	说明
颜色名称	color='red'	CSS 颜色名称, 如 red、blue、green、gray 等
单字母缩写	color='r'	'b'蓝 'g'绿 'r'红 'c'青 'm'洋红 'y'黄 'k'黑 'w'白
Hex 字符串	color='#7B2FBE'	六位十六进制, 前两位 R, 中两位 G, 后两位 B
RGB 元组	color=(0.48, 0.18, 0.74)	三个 0~1 之间的浮点数, 依次为 R、G、B
RGBA 元组	color=(0.48, 0.18, 0.74, 0.5)	第四个值为透明度 (alpha)

```
plt.plot(x, y, color='steelblue')    # 颜色名称
plt.plot(x, y, color='#4B2E84')     # Hex
plt.plot(x, y, color=(0.29, 0.18, 0.52)) # RGB, 值域 0~1
plt.plot(x, y, color='red', alpha=0.3) # alpha 控制整条线的透明度
```

5.5 颜色设计原则：避免高饱和度对比色

颜色饱和度的公式： $\text{saturation} = (\max(R,G,B) - \min(R,G,B)) / \max(R,G,B)$ 。饱和度趋近 1 的纯色（如纯红、纯绿、纯蓝）会在视网膜上产生「色差像差」，长时间盯着容易眼疲劳，信息传递效率反而下降。

- **避免**：纯红(255,0,0)、纯绿(0,255,0)、纯蓝(0,0,255)等全饱和色
- **推荐**：在 RGB 通道中混入灰度（即让最小通道值 > 0），例如 #4B2E84（暗紫）比 #7F00FF（纯紫）舒适得多
- **工具**：使用 ColorBrewer (<https://colorbrewer2.org/>) 按数据类型（顺序/发散/分类）选调色板

5.6 颜色映射 (Colormap)

当颜色需要编码连续数值时，使用 Colormap（调色板）比手动指定颜色更科学。`plt.scatter()` 支持 `cmap` 参数，但 `plt.plot()` 不支持。

```
# plt.scatter 支持 cmap, c 参数指定用于映射颜色的数值
plt.scatter(x, y, c=x, cmap='BuPu') # BuPu = 蓝-紫渐变
plt.colorbar() # 显示颜色条（对应数值范围）
plt.show()
```

常用 Colormap 按类型分类：

类型	适用场景	推荐名称
顺序型 (Sequential)	从低到高的单调数值	Blues, BuPu, YlOrRd, viridis, plasma
发散型 (Diverging)	以中间值为基准的正负偏差	RdBu, PuOr, coolwarm, bwr
定性型 (Qualitative)	分类数据，无大小顺序	Paired, Set2, tab10, tab20

5.7 格式字符串 (Format String)

当只需要简单设置标记+线型+颜色时，可以用格式字符串（第三个位置参数）代替关键字参数，格式为 `[marker][line][color]`，顺序可以互换，每部分均可省略。

```
plt.plot(x, y, 'o') # 仅圆点标记，无线条
plt.plot(x, y, '--') # 虚线，无标记
plt.plot(x, y, 'k') # 黑色实线
plt.plot(x, y, 'o--k') # 圆点 + 虚线 + 黑色
plt.plot(x, y, 'r^-.') # 红色上三角 + 点划线

# 注意：x、y 和格式字符串均为位置参数，不能用关键字形式传递
# 正确：plt.plot(x, y, 'o--k')
# 错误：plt.plot(x, y, fmt='o--k') # 会报错
```

5.8 文字大小

matplotlib 的文字大小单位为点 (point)。可在各函数中用 `fontsize=` 单独设置：

```
plt.plot(x, y)
plt.title('Sine Wave', fontsize=16) # 标题
plt.xlabel('x', fontsize=14) # x 轴标签
plt.ylabel('y', fontsize=14) # y 轴标签
plt.xticks(fontsize=12) # x 轴刻度数字
plt.yticks(fontsize=12) # y 轴刻度数字
plt.show()
```

```
# 全局设置字体大小（影响此后所有图）
plt.rcParams['font.size'] = 14
```

5.9 添加图例

在每条线上加 `label=` 参数，再调用 `plt.legend()`，即可自动生成图例：

```
plt.plot(x, np.sin(x),      label='Sin(x)')
plt.plot(x, np.sin(2*x),    label='Sin(2x)')
plt.plot(x, np.cos(x),      label='Cos(x)',  linestyle='--')
plt.legend()                # 自动从 label 参数生成图例
plt.legend(loc='upper right') # 手动指定位置
plt.legend(loc='best')       # 自动选最佳位置（默认）
plt.show()
```

6 子图布局 (Subplots)

子图是指在一个 Figure 画布上排列多个 Axes 绘图区，适合对比展示多个相关图形。

6.1 基本子图创建

```
# 创建 3 行 1 列的子图
fig, axes = plt.subplots(nrows=3, ncols=1, layout='constrained')
# axes 是形状为 (3,) 的 ndarray, 用 axes[i] 访问

for i in range(3):
    axes[i].plot(x, np.sin((i+1)*x))
    axes[i].set_title(f'Sin({i+1}x)')

fig.suptitle('Sine Functions') # 整个 Figure 的总标题
plt.show()
```

```
# 创建 2 行 2 列的子图
fig, axes = plt.subplots(nrows=2, ncols=2)
# axes 是形状为 (2, 2) 的 ndarray, 用 axes[i, j] 访问

for i in range(2):
    for j in range(2):
        axes[i, j].plot(x, np.sin((2*i+j+1)*x))
        axes[i, j].set_title(f'Sin({2*i+j+1}x)')

fig.suptitle('Sine Functions')
```

```
fig.tight_layout()    # 自动调整间距，防止标题重叠
plt.show()
```

6.2 间距控制：tight_layout vs constrained_layout

fig.tight_layout()	layout='constrained'
事后调用	创建时传入参数
手动触发，可在任意时刻调用	自动持续生效
老接口，兼容性更好	新接口，效果更稳定
fig.tight_layout()	plt.subplots(..., layout='constrained')

6.3 手动调整边距 (subplots_adjust)

当 tight_layout 效果不理想时，可以用 `fig.subplots_adjust()` 精细控制边距：

```
fig, axes = plt.subplots(nrows=2, ncols=2, sharex=True, sharey=True)

for i in range(2):
    for j in range(2):
        axes[i, j].plot(x, np.sin((2*i+j+1)*x))
        # 在坐标 (2π-1, 0.9) 处添加文字标注 (x 坐标, y 坐标)
        axes[i, j].text(2*np.pi-1, 0.9, f'Sin({2*i+j+1}x)', size=8)

fig.suptitle('Sine Functions')
fig.subplots_adjust(
    hspace=0,          # 子图间垂直间距 (0 = 紧靠)
    wspace=0,          # 子图间水平间距 (0 = 紧靠)
    bottom=0.06,       # 最底部子图到 Figure 底边的距离 (比例)
    top=0.94,          # 最顶部子图到 Figure 顶边的距离 (比例)
)
plt.show()
```

各参数含义（所有值均为 0~1 的比例，相对于 Figure 尺寸）：

参数	含义
left	所有子图左边界到 Figure 左边的距离
right	所有子图右边界到 Figure 右边的距离
bottom	所有子图下边界到 Figure 底边的距离
top	所有子图上边界到 Figure 顶边的距离

参数	含义
hspace	子图行与行之间的垂直间距（占子图平均高度的比例）
wspace	子图列与列之间的水平间距（占子图平均宽度的比例）

```
# sharex / sharey: 共享坐标轴范围（对比图时非常有用）
fig, axes = plt.subplots(2, 2, sharex=True, sharey=True)
# sharex=True: 所有子图共享同一 x 轴范围，缩放时联动
# sharey=True: 所有子图共享同一 y 轴范围
```

7 常用统计图类型

下面汇总五种最常用的统计图，分别介绍其适用场景和基本用法（matplotlib 原生版本）：

图类型	适用场景	matplotlib 函数
折线图（Line Plot）	展示连续变量随另一变量的变化趋势；时间序列	plt.plot(x, y)
散点图（Scatter Plot）	探索两个连续变量之间的关联、分布和离群点	plt.scatter(x, y)
柱状图（Bar Chart）	比较各类别的数量或统计量（均值、总和等）	plt.bar(x, height)
直方图（Histogram）	观察单个连续变量的分布形状（频率/频数）	plt.hist(data)
箱线图（Box Plot）	展示分布的五数概括，快速发现离群点和偏斜	plt.boxplot(data)

7.1 折线图

```
plt.plot(x, np.sin(x))
plt.title('Sine Wave')
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.show()
```

7.2 散点图

```
# matplotlib 原生散点图
plt.scatter(data['Unit_cost'], data['AI_value'])
plt.xlabel('Unit Cost')
plt.ylabel('AI Value')
plt.show()
```

7.3 柱状图

```
# matplotlib 原生柱状图
plt.bar(data['Unit_name'], data['AI_value'])
plt.xticks(rotation=45, ha='right') # 旋转 x 轴标签防重叠
plt.tight_layout()
plt.show()
```

7.4 直方图

```
# matplotlib 原生直方图
plt.hist(data['AI_value'], bins=20) # bins 控制柱子数量
plt.xlabel('AI Value')
plt.ylabel('Frequency')
plt.show()
```

7.5 箱线图

```
# matplotlib 原生箱线图
plt.boxplot(data['AI_value'])
plt.ylabel('AI Value')
plt.show()
```

8 Seaborn：更强大的统计绘图

Seaborn 是基于 matplotlib 的高层统计绘图库，它解决了 matplotlib 原生函数在处理分组数据和 DataFrame 时的繁琐之处。

- **直接接受 DataFrame**，通过列名指定 x/y 变量，无需提前取列
- **内置分组映射**：hue（颜色）、style（线型/标记）、size（大小）参数，一行代码完成分组可视化
- **更好的默认样式**：更现代的配色方案、更合理的字体大小、更专业的图例

- 与 matplotlib 无缝兼容: seaborn 绘图后, 所有 matplotlib 的 plt/ax 命令仍然有效

```
import seaborn as sns
import matplotlib.pyplot as plt
```

8.1 散点图 (scatterplot)

```
# 基础用法: 指定 DataFrame、x 列名、y 列名
sns.scatterplot(data, x='Unit_cost', y='AI_value')
plt.show()

# 高级用法: 同时映射颜色 (hue)、形状 (style)、调色板 (palette)
fig, ax = plt.subplots(figsize=(8, 8), layout='constrained')
sns.scatterplot(
    data,
    x='Unit_cost',
    y='AI_value',
    ax=ax,                # 指定绘制在哪个 Axes 上
    hue='Level',          # 用颜色区分 Level 列
    style='Faction',       # 用形状区分 Faction 列
    s=100,                # 标记大小 (点的面积)
    palette='Paired',      # ColorBrewer 调色板名称
)
plt.show()
```

8.2 折线图 (lineplot)

```
# 当 x 有多个 y 值时, seaborn 自动计算均值并绘制置信区间
sns.lineplot(data, x='Unit_cost', y='AI_value', hue='Faction')
plt.show()

# errorbar='sd' 将置信区间改为均值 ± 标准差
sns.lineplot(data, x='Level', y='AI_value', errorbar='sd')
plt.show()
```

8.3 柱状图 (barplot)

```
# 自动计算各组均值, 并添加误差棒 (默认 95% CI)
sns.barplot(data, x='Level', y='AI_value', capsize=.2)
plt.show()

# 分组柱状图: hue 参数自动生成分组
fig, ax = plt.subplots(figsize=(6, 6), layout='constrained')
sns.barplot(
    data,
```

```

    y='Faction',          # 类别放在 y 轴（水平柱状图）
    x='AI_value',
    ax=ax,
    hue='Level',          # 按 Level 分组着色
    orient='y',           # 水平方向
    palette='Paired',
)
ax.set_xlim(0, 11000)    # 手动设置 x 轴范围
plt.show()

```

8.4 直方图 (histplot)

```

# 基础直方图
sns.histplot(data, x='AI_value')
plt.show()

# 指定箱数 + 叠加 KDE（核密度估计）曲线
sns.histplot(data, x='AI_value', bins=8, kde=True)
plt.show()

```

8.5 箱线图 (boxplot)

```

fig, ax = plt.subplots(figsize=(8, 6), layout='constrained')
sns.boxplot(
    data,
    x='Faction',          # 各 Faction 画一个箱线图
    y='AI_value',
    width=0.6,            # 控制箱体宽度 (0~1)
)
plt.show()

```

如何读懂箱线图

箱体 (Box)：下边 = Q1（第一四分位数），上边 = Q3（第三四分位数），中线 = 中位数 (Q2)

IQR（四分位距）= $Q3 - Q1$ ，反映数据的中间 50% 的离散程度

须 (Whisker)：延伸至 $Q1 - 1.5 \times IQR$ 和 $Q3 + 1.5 \times IQR$ 范围内的最极端值 (Inner Fence)

疑似异常值 (Suspected Outlier)：超过 $1.5 \times IQR$ 但不超过 $3 \times IQR$ 的点，用空心圆标出

异常值 (Outlier)：超过 $3 \times IQR$ 的点 (Outer Fence 之外)，用实心点标出

9 Seaborn 分面绘图 (Faceting)

分面 (Faceting) 是指按照某个分类变量的不同取值，**自动将数据拆分并生成多个子图**，每个子图对应该变量的一个水平 (Facet)。这比手动循环创建子图高效得多。

Seaborn 将分面功能集中在三个「Figure-level」函数中（区别于只操作单个 Axes 的「Axes-level」函数）：

函数	支持的图类型	说明
<code>sns.relplot()</code>	scatter、line	Relational Plot，适合展示变量间关系
<code>sns.catplot()</code>	box、bar、strip、violin 等	Categorical Plot，适合分类数据
<code>sns.displot()</code>	hist、kde、ecdf	Distribution Plot，适合分布可视化

9.1 relplot()

通过 `col=` 和 `row=` 参数指定分面维度：

```
# 读取数据
data = pd.read_csv('salaries.csv').convert_dtypes()

# 按 sex 分列（两列子图），用颜色区分 rank
sns.relplot(
    data,
    x='yrs.service',
    y='salary',
    kind='scatter',    # 或 'line'
    hue='rank',
    col='sex',          # 按 sex 的取值拆分为多列
)
plt.show()

# 同时按 rank 分列、sex 分行（形成网格子图）
sns.relplot(
    data,
    x='yrs.service', y='salary',
    kind='scatter',
    col='rank',          # 列方向分面
    row='sex',           # 行方向分面
)
plt.show()
```

9.2 catplot()

适合分类变量的分面图，支持箱线图、柱状图、条带图等多种子图类型：

```
# 按 sex 分面的箱线图
sns.catplot(
    data,
    x='rank',
    y='salary',
    kind='box',
    hue='rank',          # 用颜色区分 rank (与 x 相同时起到着色作用)
    width=0.5,
    col='sex',
)
plt.show()

# 按 sex 分面的柱状图 (先用 groupby 计算均值)
tmp = data.groupby(['rank', 'sex'])['salary'].mean().reset_index()
sns.catplot(
    tmp,
    x='rank',
    y='salary',
    kind='bar',
    hue='rank',
    col='sex',
)
plt.show()
```

9.3 displot()

适合分布图的分面，支持直方图、KDE（核密度估计）等：

```
# — 直方图分面 —————

# 按 rank 分列
sns.displot(data, x='salary', kind='hist', col='rank')
plt.show()

# 堆叠直方图 (按 rank 着色并堆叠在同一子图)
sns.displot(data, x='salary', kind='hist', hue='rank', multiple='stack')
plt.show()

# — 密度图分面 —————

# 按 sex 分列
sns.displot(data, x='salary', kind='kde', col='sex')
plt.show()

# 用颜色区分 sex (同一子图内叠加两条密度曲线)
sns.displot(data, x='salary', kind='kde', hue='sex')
plt.show()
```

Figure-level vs Axes-level 的区别

Axes-level 函数（`sns.scatterplot`、`sns.boxplot` 等）：

- 返回一个 Axes 对象，可通过 `ax=` 参数指定绘制位置
- 可与 matplotlib subplots 结合，手动拼接复杂布局

Figure-level 函数（`sns.relplot`、`sns.catplot`、`sns.displot`）：

- 自动创建 Figure 和所有子图（不接受 `ax=` 参数）
- 返回 FacetGrid 对象，通过 `.set_axis_labels()`、`.set_titles()` 等方法调整
- 不要在它之后调用 `plt.subplots_adjust()` — 用 FacetGrid 的方法代替

10 Plotnine: Python 的 ggplot2

Plotnine 是 R 语言中鼎鼎大名的 `ggplot2` 包的 Python 移植版，基于 Leland Wilkinson 的「图形语法（Grammar of Graphics）」理论设计。

10.1 图形语法的核心理念

图形语法将一张图分解为若干独立的「层」，每层负责不同的视觉职责。组合多个层，便构成完整的图形：

组件	含义	举例
Data	输入的数据框	salaries 数据集
Aes	美学映射（变量→视觉）	<code>x='salary', color='sex'</code>
Geom	几何对象（图形类型）	<code>geom_point</code> , <code>geom_boxplot</code> , <code>geom_density</code>
Stat	统计变换	<code>stat_bin</code> （直方图）, <code>stat_smooth</code> （平滑曲线）
Scale	尺度（映射规则）	<code>scale_color_brewer</code> , <code>scale_x_log10</code>
Facet	分面	<code>facet_grid</code> , <code>facet_wrap</code>
Theme	主题（非数据外观）	<code>theme_classic</code> , <code>theme_bw</code>

10.2 语法结构

Plotnine 的图用 `+` 运算符将各层叠加，结构非常清晰：

```
from plotnine import *
```

```

# — 密度图（按性别分组填色）——
(
  ggplot(data, aes(x='salary', fill='sex', color='sex'))
  + geom_density(alpha=0.3)    # alpha 控制填充透明度
)

# — 箱线图（按性别着色）——
(
  ggplot(data, aes(x='rank', y='salary', color='sex'))
  + geom_boxplot()
)

# — 直方图 + 按 rank 分面（行方向）——
(
  ggplot(data, aes(x='salary'))
  + geom_histogram(fill='blue', alpha=0.3)
  + facet_grid(rows='rank')    # rows= 指定行分面变量
)

```

10.3 与 seaborn 的对比

维度	Seaborn	Plotnine
设计哲学	函数式，每种图有独立函数	声明式，所有图用相同语法组合
学习曲线	上手容易，参数直觉	需要理解图形语法，初学稍难
灵活性	中等	极高，层可任意叠加
底层引擎	matplotlib	matplotlib（间接）
背景	Python 原生	R 的 ggplot2 移植
推荐人群	Python 为主，喜欢简单快速	熟悉 R/ggplot2，或追求代码表达力

11 综合速查表

11.1 plt 接口 vs ax 接口

操作	plt 接口	ax 接口
绘折线	plt.plot(x, y)	ax.plot(x, y)
绘散点	plt.scatter(x, y)	ax.scatter(x, y)

操作	plt 接口	ax 接口
设标题	plt.title('...')	ax.set_title('...')
设 x 标签	plt.xlabel('...')	ax.set_xlabel('...')
设 y 标签	plt.ylabel('...')	ax.set_ylabel('...')
设 x 范围	plt.xlim(a, b)	ax.set_xlim(a, b)
设 y 范围	plt.ylim(a, b)	ax.set_ylim(a, b)
加图例	plt.legend()	ax.legend()
加网格	plt.grid()	ax.grid()
加文字	plt.text(x, y, '...')	ax.text(x, y, '...')
保存图	plt.savefig('xxx.png')	fig.savefig('xxx.png')
关闭图	plt.close()	plt.close(fig)

11.2 seaborn 函数速查

图类型	Axes-level 函数	Figure-level (分面)
散点图	sns.scatterplot()	sns.relplot(..., kind='scatter')
折线图	sns.lineplot()	sns.relplot(..., kind='line')
柱状图	sns.barplot()	sns.catplot(..., kind='bar')
箱线图	sns.boxplot()	sns.catplot(..., kind='box')
直方图	sns.histplot()	sns.displot(..., kind='hist')
密度图	sns.kdeplot()	sns.displot(..., kind='kde')
散点矩阵	—	sns.pairplot()
热图	sns.heatmap()	—

11.3 常见问题排查

症状	原因	解决方法
图上多了一条线	没有 plt.close() 就重新绘图	每张图结束后调用 plt.close()
子图标题重叠	没有调整间距	fig.tight_layout() 或 layout='constrained'
保存图有空白	没用 bbox_inches='tight'	fig.savefig('x.png', bbox_inches='tight')

症状	原因	解决方法
中文显示方块	字体不支持中文	<code>plt.rcParams['font.sans-serif'] = ['SimHei']</code>
y 轴范围自动变	多次 <code>plt.plot()</code> 叠加在同一图	在第一次 <code>plot</code> 前调用 <code>plt.figure()</code> 或 <code>plt.close()</code>
保存 PNG 模糊	DPI 太低	<code>fig.savefig('x.png', dpi=300)</code>

「好的可视化让数据开口说话，坏的可视化让数据闭嘴撒谎。」

—— 课件最后一句话